

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №1

Вариант А 27

Выполнил:
студент группы ИУ5 - 32Б:
Акопов А.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.
Подпись и дата:

Москва, 2025 г.

Задание:

Рубежный контроль представляет собой разработку программы на языке Python, которая выполняет следующие действия:

1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко-многим.

Пример классов данных для предметной области Сотрудник-Отдел:

1. Класс «Сотрудник», содержащий поля:

- ID записи о сотруднике;
- Фамилия сотрудника;
- Зарплата (количественный признак);
- ID записи об отделе. (для реализации связи один-ко-многим)

2. Класс «Отдел», содержащий поля:

- ID записи об отделе;
- Наименование отдела.

3. (Для реализации связи многие-ко-многим) Класс «Сотрудники отдела», содержащий поля:

- ID записи о сотруднике;
- ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5

записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом. Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в

примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При

разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много»,

произвольный количественный признак, например, «зарплата сотрудника».

Результатом рубежного контроля является документ в формате PDF, который содержит

текст программы и результаты ее выполнения.

Вариант А.

1. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список всех связанных сотрудников и отделов, отсортированный по отделам,

сортировка по сотрудникам произвольная.

2. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список отделов с суммарной зарплатой сотрудников в каждом отделе, отсортированный по суммарной зарплате.

3. «Отдел» и «Сотрудник» связаны соотношением многие-ко-многим. Выведите список всех отделов, у которых в названии присутствует слово «отдел», и список работающих в них сотрудников.

Код:

```
from operator import itemgetter

class Teacher:
    """Преподаватель"""
    def __init__(self, id, surname, salary, course_id):
        self.id = id
        self.surname = surname
        self.salary = salary
        self.course_id = course_id

class Course:
    """Учебный курс"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class TeacherCourse:
    """Преподаватели курсов (для связи многие-ко-многим)"""
    def __init__(self, teacher_id, course_id):
        self.teacher_id = teacher_id
        self.course_id = course_id

courses = [
    Course(1, 'Математический анализ'),
    Course(2, 'Программирование на Python'),
    Course(3, 'Базы данных'),
    Course(4, 'Алгоритмы и структуры данных'),
    Course(5, 'Физика'),
]

teachers = [
    Teacher(1, 'Иванов', 50000, 1),
    Teacher(2, 'Петров', 60000, 2),
    Teacher(3, 'Сидоров', 55000, 3),
    Teacher(4, 'Кузнецов', 65000, 4),
    Teacher(5, 'Попов', 52000, 5),
    Teacher(6, 'Смирнов', 58000, 1),
]
```

```

    Teacher(7, 'Федоров', 62000, 2),
]

teachers_courses = [
    TeacherCourse(1, 1),
    TeacherCourse(1, 3),
    TeacherCourse(2, 2),
    TeacherCourse(3, 3),
    TeacherCourse(4, 4),
    TeacherCourse(5, 5),
    TeacherCourse(6, 1),
    TeacherCourse(7, 2),
    TeacherCourse(2, 4),
]

def main():
    """Основная функция"""

    # Соединение данных один-ко-многим
    one_to_many = [(t.surname, t.salary, c.name)
                    for c in courses
                    for t in teachers
                    if t.course_id == c.id]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [(c.name, tc.course_id, tc.teacher_id)
                          for c in courses
                          for tc in teachers_courses
                          if c.id == tc.course_id]

    many_to_many = [(t.surname, t.salary, course_name)
                     for course_name, course_id, teacher_id in many_to_many_temp
                     for t in teachers if t.id == teacher_id]

    print('Задание A1')
    res_1 = sorted(one_to_many, key=itemgetter(2))
    for i in res_1:
        print(f'Курс: {i[2]}, Преподаватель: {i[0]}, Зарплата: {i[1]}')

    print('\nЗадание A2')
    res_2_unsorted = []
    for c in courses:
        course_teachers = list(filter(lambda i: i[2] == c.name, one_to_many))
        # Если на курсе есть преподаватели
        if len(course_teachers) > 0:
            # Суммируем зарплаты всех преподавателей курса
            total_salary = [salary for _, salary, _ in course_teachers]
            total_salary_sum = sum(total_salary)
            res_2_unsorted.append((c.name, total_salary_sum))

```

```

# Сортируем по суммарной зарплате (по убыванию)
res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
for course_name, total_salary in res_2:
    print(f'Курс: {course_name}, Суммарная зарплата: {total_salary}')

print('\nЗадание A3')
res_3 = {}

for c in courses:

    if 'данных' in c.name:

        course_teachers = list(filter(lambda i: i[2] == c.name,
many_to_many))
        teacher_surnames = [surname for surname, _, _ in course_teachers]
        res_3[c.name] = teacher_surnames

    for course_name, teachers_list in res_3.items():
        teachers_str = ', '.join(teachers_list) if teachers_list else 'нет
преподавателей'
        print(f'Курс: {course_name}, Преподаватели: {teachers_str}')

if __name__ == "__main__":
    main()

```

Скриншот работы:

```

(base) an.akorov@MacBook-Air-Akorov RK % python3 RK1.py
Задание A1
Курс: Алгоритмы и структуры данных, Преподаватель: Кузнецов, Зарплата: 65000
Курс: Базы данных, Преподаватель: Сидоров, Зарплата: 55000
Курс: Математический анализ, Преподаватель: Иванов, Зарплата: 50000
Курс: Математический анализ, Преподаватель: Смирнов, Зарплата: 58000
Курс: Программирование на Python, Преподаватель: Петров, Зарплата: 60000
Курс: Программирование на Python, Преподаватель: Федоров, Зарплата: 62000
Курс: Физика, Преподаватель: Попов, Зарплата: 52000

Задание A2
Курс: Программирование на Python, Суммарная зарплата: 122000
Курс: Математический анализ, Суммарная зарплата: 108000
Курс: Алгоритмы и структуры данных, Суммарная зарплата: 65000
Курс: Базы данных, Суммарная зарплата: 55000
Курс: Физика, Суммарная зарплата: 52000

Задание A3
Курс: Базы данных, Преподаватели: Иванов, Сидоров
Курс: Алгоритмы и структуры данных, Преподаватели: Кузнецов, Петров
(base) an.akorov@MacBook-Air-Akorov RK %

```